

Author: Owen Lindsey
Instructor: Professor David Parker
Course: CST-180: Python Programming 1
Institution: Grand Canyon University
Date: November 09, 2025

Important Links

Video: <--- Click there

src code: <--- Click there

Summary

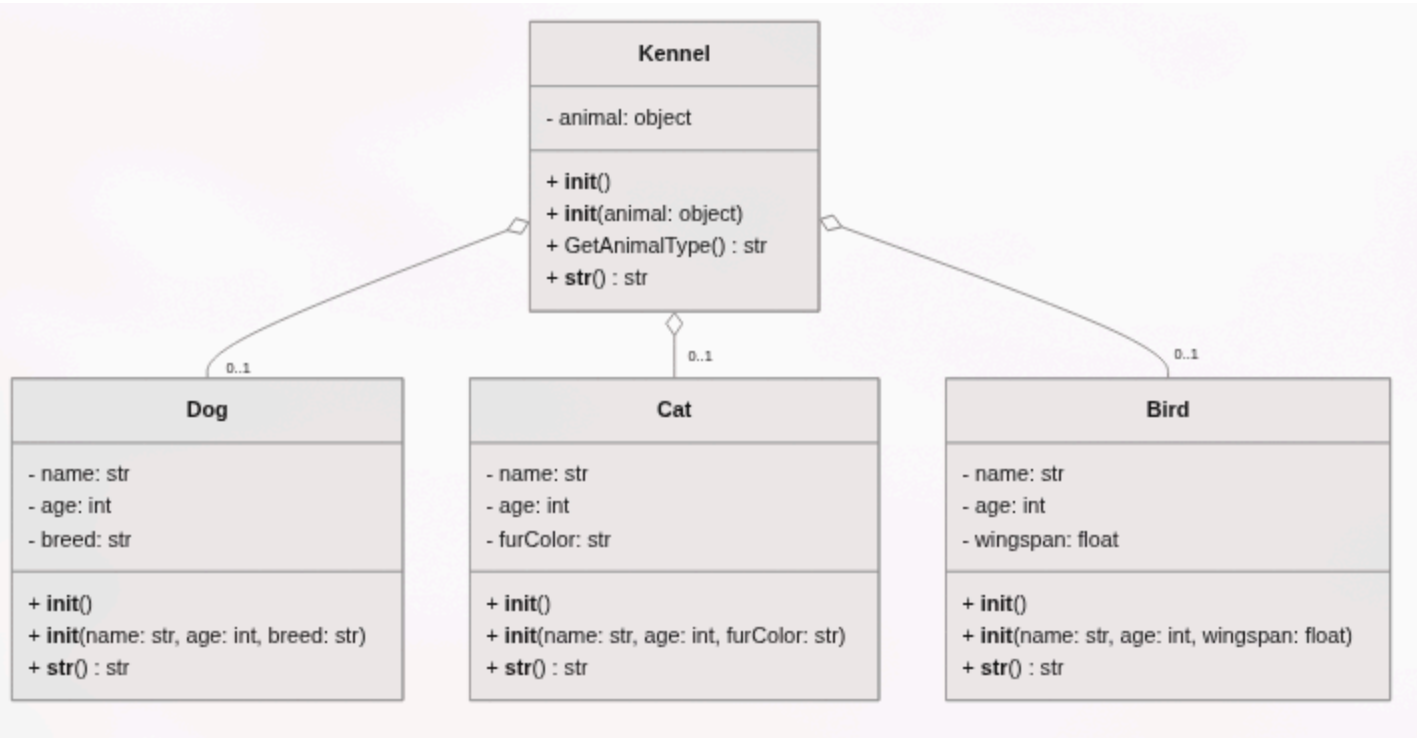
This first project establishes the object model and basic design for an animal shelter simulator without inheritance. It defines three independent animal classes (`Dog` , `Cat` , `Bird`) with required attributes and behaviors, plus a `Kennel` class that contains exactly one animal (a `Dog` , `Cat` , or `Bird`) at a time. The `Kennel` can report the contained animal's type using the object's `__class__.__name__` (i.e., the class name string). The deliverables include a UML class diagram, an IPO summary, detailed pseudocode and a flowchart for the program logic, sample data, and corresponding sample output.

Input / Processing / Output

- **Inputs:**
 - User or test-provided animal details:
 - `Dog` : Name (str), Age (int), Breed (str)
 - `Cat` : Name (str), Age (int), Fur Color (str)
 - `Bird` : Name (str), Age (int), Wingspan (float or str units)
 - Choice of which animal instance to place into a `Kennel` .
 - **Processing:**
 - Construct animal objects using an overloaded constructor (supporting both no-arg defaults and full-arg initialization).
 - Generate human-readable object text using an overloaded `__str__` method.
 - Place exactly one animal into a `Kennel` (containment).
 - Determine the contained animal type via `obj.__class__.__name__` .
 - **Outputs:**
 - Printed or displayed string representations of animals.
 - The `Kennel` string, showing which animal it contains.
 - The reported animal type from `Kennel.GetAnimalType()` .
-

UML Class Diagram (no inheritance; containment only)

Note: The diagram purposely avoids inheritance. `Kennel` has a containment relationship to one of `Dog` , `Cat` , or `Bird` .



Textual notes:

- No inheritance hierarchy is used; each animal is a standalone class.
 - `Kennel` composes/contains exactly one animal at a time (can be empty).
 - `GetAnimalType` returns the class name string of the contained object.
-

Design – Detailed Pseudocode

High-level program flow to construct animals, place them in a kennel, and report details.

```
BEGIN
  DEFINE class Dog with attributes (name, age, breed)
    PROVIDE overloaded constructors:
      - __init__() sets default values
      - __init__(name, age, breed) sets provided values
    PROVIDE __str__() that returns formatted details

  DEFINE class Cat with attributes (name, age, furColor)
    PROVIDE overloaded constructors (no-arg, full-arg)
    PROVIDE __str__()

  DEFINE class Bird with attributes (name, age, wingspan)
    PROVIDE overloaded constructors (no-arg, full-arg)
    PROVIDE __str__()

  DEFINE class Kennel with attribute (animal)
    PROVIDE overloaded constructors:
      - __init__() initializes animal to None
      - __init__(animal) sets provided Dog/Cat/Bird
    METHOD GetAnimalType():
      IF animal is None THEN return "None"
      ELSE return animal.__class__.__name__
    METHOD __str__():
      IF animal is None THEN return "Kennel(empty)"
      ELSE return "Kennel(" + animal.__str__() + ")"

  // Sample execution logic
  CREATE dog1 as Dog("Buddy", 3, "Labrador")
  CREATE cat1 as Cat("Mittens", 2, "Tabby")
  CREATE bird1 as Bird("Sky", 1, 0.35)

  CREATE kennel as Kennel()

  // Place an animal into the kennel (e.g., dog1)
  SET kennel.animal = dog1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()

  // Replace with another animal (e.g., cat1)
  SET kennel.animal = cat1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()

  // Replace with a bird
  SET kennel.animal = bird1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()
END
```

Python snippet showing expected method shapes:

```
class Dog:
    def __init__(self, name: str = "", age: int = 0, breed: str = ""):
        self.name = name
        self.age = age
        self.breed = breed
    def __str__(self) -> str:
        return f"Dog(Name={self.name}, Age={self.age}, Breed={self.breed})"

class Cat:
    def __init__(self, name: str = "", age: int = 0, furColor: str = ""):
        self.name = name
        self.age = age
        self.furColor = furColor
    def __str__(self) -> str:
        return f"Cat(Name={self.name}, Age={self.age}, FurColor={self.furColor})"

class Bird:
    def __init__(self, name: str = "", age: int = 0, wingspan: float = 0.0):
        self.name = name
        self.age = age
        self.wingspan = wingspan
    def __str__(self) -> str:
        return f"Bird(Name={self.name}, Age={self.age}, Wingspan={self.wingspan})"

class Kennel:
    def __init__(self, animal: object | None = None):
        self.animal = animal
    def GetAnimalType(self) -> str:
        return "None" if self.animal is None else self.animal.__class__.__name__
    def __str__(self) -> str:
        return "Kennel(empty)" if self.animal is None else f"Kennel({self.animal})"
```

Sample Data

Dog: Name = "Buddy", Age = 3, Breed = "Labrador"

Cat: Name = "Mittens", Age = 2, Fur Color = "Tabby"

Bird: Name = "Sky", Age = 1, Wingspan = 0.35

Sample Output (using the sample data)

```
Kennel(Dog(Name=Buddy, Age=3, Breed=Labrador))
Dog
Kennel(Cat(Name=Mittens, Age=2, FurColor=Tabby))
Cat
Kennel(Bird(Name=Sky, Age=1, Wingspan=0.35))
Bird
```

Application Output:

```
.../Homework/Animal_Shelter main • ) python animal_shelter.py
Kennel Animal: Dog: Name: Buddy, Age: 3, Breed: Labrador
Dog
Kennel Animal: Cat: Name: Mittens, Age: 2, Fur Color: Tabby
Cat
Kennel Animal: Bird: Name: Sky, Age: 1, Wingspan: 0.35
Bird

.../Homework/Animal_Shelter main • )
```


File Output:

```
.../Homework/Animal_Shelter main • ? ) python -m unittest test_animal_shelter.py
.....
-----
Ran 28 tests in 0.000s

OK

.../Homework/Animal_Shelter main • ? ) python -m unittest discover -v
test_bird_constructor_default (test_animal_shelter.TestBird.test_bird_constructor_default)
Test Bird constructor with default parameters. ... ok
test_bird_constructor_with_parameters (test_animal_shelter.TestBird.test_bird_constructor_with_parameters)
Test Bird constructor with all parameters. ... ok
test_bird_partial_parameters (test_animal_shelter.TestBird.test_bird_partial_parameters)
Test Bird constructor with partial parameters. ... ok
test_bird_str (test_animal_shelter.TestBird.test_bird_str)
Test Bird __str__ method. ... ok
test_cat_constructor_default (test_animal_shelter.TestCat.test_cat_constructor_default)
Test Cat constructor with default parameters. ... ok
test_cat_constructor_with_parameters (test_animal_shelter.TestCat.test_cat_constructor_with_parameters)
Test Cat constructor with all parameters. ... ok
test_cat_partial_parameters (test_animal_shelter.TestCat.test_cat_partial_parameters)
Test Cat constructor with partial parameters. ... ok
test_cat_str (test_animal_shelter.TestCat.test_cat_str)
Test Cat __str__ method. ... ok
test_dog_constructor_default (test_animal_shelter.TestDog.test_dog_constructor_default)
Test Dog constructor with default parameters. ... ok
test_dog_constructor_with_parameters (test_animal_shelter.TestDog.test_dog_constructor_with_parameters)
Test Dog constructor with all parameters. ... ok
test_dog_partial_parameters (test_animal_shelter.TestDog.test_dog_partial_parameters)
Test Dog constructor with partial parameters. ... ok
test_dog_str (test_animal_shelter.TestDog.test_dog_str)
Test Dog __str__ method. ... ok
test_kennel_add_bird (test_animal_shelter.TestKennel.test_kennel_add_bird)
Test adding a Bird to an empty Kennel. ... ok
test_kennel_add_cat (test_animal_shelter.TestKennel.test_kennel_add_cat)
Test adding a Cat to an empty Kennel. ... ok
test_kennel_add_dog (test_animal_shelter.TestKennel.test_kennel_add_dog)
Test adding a Dog to an empty Kennel. ... ok
test_kennel_constructor_empty (test_animal_shelter.TestKennel.test_kennel_constructor_empty)
Test Kennel constructor with no parameters. ... ok
test_kennel_constructor_with_bird (test_animal_shelter.TestKennel.test_kennel_constructor_with_bird)
Test Kennel constructor with a Bird. ... ok
test_kennel_constructor_with_cat (test_animal_shelter.TestKennel.test_kennel_constructor_with_cat)
Test Kennel constructor with a Cat. ... ok
test_kennel_constructor_with_dog (test_animal_shelter.TestKennel.test_kennel_constructor_with_dog)
Test Kennel constructor with a Dog. ... ok
test_kennel_get_animal_type_bird (test_animal_shelter.TestKennel.test_kennel_get_animal_type_bird)
Test GetAnimalType method with a Bird. ... ok
test_kennel_get_animal_type_cat (test_animal_shelter.TestKennel.test_kennel_get_animal_type_cat)
Test GetAnimalType method with a Cat. ... ok
test_kennel_get_animal_type_dog (test_animal_shelter.TestKennel.test_kennel_get_animal_type_dog)
Test GetAnimalType method with a Dog. ... ok
test_kennel_get_animal_type_empty (test_animal_shelter.TestKennel.test_kennel_get_animal_type_empty)
Test GetAnimalType method with empty kennel. ... ok
test_kennel_replace_animal (test_animal_shelter.TestKennel.test_kennel_replace_animal)
Test replacing an animal in the Kennel (only one animal at a time). ... ok
test_kennel_str_empty (test_animal_shelter.TestKennel.test_kennel_str_empty)
Test Kennel __str__ method when empty. ... ok
test_kennel_str_with_bird (test_animal_shelter.TestKennel.test_kennel_str_with_bird)
Test Kennel __str__ method with a Bird. ... ok
test_kennel_str_with_cat (test_animal_shelter.TestKennel.test_kennel_str_with_cat)
Test Kennel __str__ method with a Cat. ... ok
test_kennel_str_with_dog (test_animal_shelter.TestKennel.test_kennel_str_with_dog)
Test Kennel __str__ method with a Dog. ... ok

-----
Ran 28 tests in 0.000s

OK
```

Test Output:

```
Neo-tree
/run/media/omniv/T7/GCU_SWE_2023-2025
> AIT-104-Data-mining-machine-learning
> CST-150-C#1
v CST-180-Python
  v Homework
    v AnimalShelter
      > __pycache__
      AnimalShelterPseudocode.md
      AnimalShelterPseudocode.pdf
      animal_shelter.md
      animal_shelter.py
      animal_shelter_p1.py
      animal_shelter_pt2.md
      animal_shelter_pt2.pdf
      test_animal_shelter.py
    PopulationSimulation
    TheologicalFoundations
    War Card Game
    input-output
    NeoVimPractice.py
    (1 hidden item)
  Photos
  Templates
  Zybook_notes
  class notes
  plugins
  snippets
  themes
  Pasted image 20251106094822.png
  Pasted image 20251106105418.png
  Pasted image 20251106145000.png?
  Pasted image 20251106145210.png?
  README.md
  app.json
  appearance.json
  community-plugins.json
  core-plugins.json
  graph.json
  workspace.json
  (1 hidden item)
> CST-201-algorithms-data-structures
> CST-239-Java2
> CST-250-C#2
> CST-321-Operating-system-fundamentals
> CST-326-Written-Verbal-Communication
> CST-339-Java3
> CST-345-Database-Design
> CST-350-C#3
> CST-391-Web_dev
> CST-407-Application-Security-Fundamentals
> SummerPractice
  LICENSE
  README.md
  orders-response.json
  swagger-spec.json
  (4 hidden items)
```


Resources

Dalbey, J. (2003). *Pseudocode Standard*. Retrieved from https://users.csc.calpoly.edu/~jdalbey/SWE/pdl_std.html